

MODULAR MULTI-AGENT DOCUMENT AI PLATFORM: ADAPTIVE POLICY ROUTING AND EXPLAINABLE DOCUMENT INTELLIGENCE

25-26J-129

Final Report

Dasanayake D.R.L, Nayanapriya S.A.T,
Wijetunga W.L.A.T, Pathirana M.M.

BSc (Hons) in Information Technology Specializing in Data Science

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

**MODULAR MULTI-AGENT DOCUMENT AI PLATFORM:
ADAPTIVE POLICY ROUTING AND EXPLAINABLE DOCUMENT
INTELLIGENCE**

25-26J-129

Final Report

BSc (Hons) in Information Technology Specializing in Information
Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature

The above candidates have carried out research for the bachelor's degree Dissertation under my supervision.

.....

Signature of Supervisor

Mr. Samadhi Rathnayake

.....

Date

.....

Signature of Co Supervisor

Dr. Lakmini Abeywaradena

.....

Date

ABSTRACT

Document intelligence systems face the dual challenge of processing diverse document types adaptively while ensuring that extracted outputs are accurate and verifiable. This dissertation presents a modular, multi-agent document AI platform designed to address these challenges through four tightly integrated components. First, an Adaptive Hierarchical Memory Network (AHMN) employs Thompson Sampling with Beta-Bernoulli posteriors for policy selection and introduces two novel mechanisms: Retrieval-Augmented Bandit Initialization (RABI) for cold-start knowledge transfer, and Drift-Aware Adaptive Forgetting (DAAF) for handling concept drift in document distributions. Second, an Intelligent Document Processing Orchestrator (IDPO) executes a thirteen-agent pipeline covering optical character recognition, preprocessing, LLM-based field extraction, validation, and recovery. Third, a six-layer Guardrail Ensemble performs hallucination detection and output validation using schema adherence, relevance scoring, internal consistency checking, domain rules, PII leakage detection, and faithfulness scoring. Fourth, an Explainable AI system with Human-in-the-Loop (XAI-HITL) feedback provides BART-based generative question answering supported by cross-attention and LIME-based explanations. The entire platform is deployed as eight Docker microservices with PostgreSQL and pgvector for persistent storage. Results demonstrate reliable document extraction across multiple document types including invoices, clinical summaries, receipts, bank statements, and insurance claims, with the guardrail ensemble preventing hallucinated content from reaching end users and the XAI-HITL component enabling continuous model monitoring.

Keywords:

multi-agent systems, document intelligence, Thompson Sampling, hallucination detection, explainable AI, human-in-the-loop

ACKNOWLEDGEMENTS

The authors wish to express their sincere gratitude to the academic staff of the Department of Information Technology at Sri Lanka Institute of Information Technology for their guidance and sustained support throughout this research project.

Special thanks are extended to the project supervisor for providing continuous feedback and direction that shaped the design and implementation of this platform. The team gratefully acknowledges the open-source communities behind FastAPI, HuggingFace Transformers, PaddleOCR, SentenceTransformers, and Docker, whose tools formed the technical foundation of this work.

Finally, the authors thank their families for their patience and encouragement throughout the duration of this project.

TABLE OF CONTENTS

1. INTRODUCTION	11
1.1 Background and Literature Survey	11
1.2 Research Gap	14
1.3 Research Problem	15
1.4 Research Objectives	15
2. METHODOLOGY	16
2.1 System Architecture Overview	16
2.2 Database Design	17
2.3 Adaptive Hierarchical Memory Network (AHMN)	18
2.3.1 Three-Tier Memory Architecture	19
2.3.3 RABI: Retrieval-Augmented Bandit Initialization	20
2.3.4 DAAF: Drift-Aware Adaptive Forgetting	21
2.4 Intelligent Document Processing Orchestrator (IDPO)	22
2.4.1 Thirteen-Agent Pipeline	22
2.4.2 Retrieval System	24
2.4.3 Provenance and Audit Trail	25
2.5 Guardrail Ensemble	25
2.5.1 Schema Adherence	26
2.5.2 Relevance Check	26
2.5.3 Internal Consistency	26
2.5.4 Domain Rules	27
2.5.5 PII Leakage Detection	27
2.5.6 Faithfulness Scoring	27
2.6 Explainable AI and Human-in-the-Loop (XAI-HITL)	28
2.6.1 BART-based Generative Question Answering	28
2.6.2 Question Type Handling	30
2.6.3 Explainability Mechanisms	30
2.6.4 HITL Feedback System	31
2.7 API Gateway	32
2.8 Frontend Dashboard	32
2.9 Deployment and Infrastructure	33

3. RESULTS AND DISCUSSION	34
3.1 System Integration	34
3.2 AHMN Policy Adaptation	34
3.3 Guardrail Validation Coverage	35
3.4 XAI-HITL Feedback Loop	36
3.5 Summary of Each Student's Contribution	36
3.6 Discussion	37
4.1 Limitations	40
4.2 Future Work	40
REFERENCE	42
APPENDICES	44

LIST OF TABLES

Table 1 - Docker service resource allocations	16
Table 2 -Database tables and their purpose.....	18
Table 3 - AHMN three-tier memory architecture	19
Table 4 - Thompson Sampling posterior update rules	20
Table 5 - RABI cold-start transfer parameters	21
Table 6 - DAAF drift detection parameters	22
Table 7 - IDPO thirteen-agent pipeline	23
Table 8 - Guardrail ensemble validation layers	25
Table 9 - Faithfulness scoring component weights.....	28
Table 10 - BART model configuration parameters.....	29
Table 11 - XAI-HITL question type handling	30
Table 12 - ML models used across services.....	33

LIST OF ABBREVIATIONS

AHMN	Adaptive Hierarchical Memory Network
API	Application Programming Interface
BART	Bidirectional and Auto-Regressive Transformer
BM25	Best Match 25 (term-frequency ranking function)
CPU	Central Processing Unit
DAAF	Drift-Aware Adaptive Forgetting
HITL	Human-in-the-Loop
IDPO	Intelligent Document Processing Orchestrator
JWT	JSON Web Token
LIME	Local Interpretable Model-agnostic Explanations
LLM	Large Language Model
LTM	Long-Term Memory
MTM	Mid-Term Memory
NER	Named Entity Recognition
OCR	Optical Character Recognition
PII	Personally Identifiable Information
RABI	Retrieval-Augmented Bandit Initialization
Redis	Remote Dictionary Server
REST	Representational State Transfer
SHAP	SHapley Additive explanations
SLIIT	Sri Lanka Institute of Information Technology
STM	Short-Term Memory
XAI	Explainable Artificial Intelligence

1. INTRODUCTION

1.1 Background and Literature Survey

The processing of digital documents has become a critical operational requirement across industries ranging from healthcare and finance to logistics and legal services. Modern organizations deal with large volumes of heterogeneous documents including invoices, medical records, bank statements, and purchase orders, each presenting different structural and semantic challenges for automated extraction systems. The field of Document AI has advanced rapidly in recent years, driven by breakthroughs in deep learning and large language models, yet practical deployment still faces significant obstacles related to adaptability, accuracy, and trustworthiness.

One of the central challenges in document processing is the selection of an appropriate processing strategy for a given document. Early rule-based systems applied fixed pipelines that could not adapt to variations in document layout, vendor format, or content quality [7]. Machine learning approaches improved flexibility but typically required large labelled datasets and lacked mechanisms for online adaptation as document distributions shifted over time. The concept of policy-driven processing, where a system learns which processing strategy works best for each document class, represents a more principled approach to this problem.

Multi-armed bandit algorithms provide a formal framework for online decision-making under uncertainty, and Thompson Sampling in particular has proven to be an effective strategy for balancing exploration and exploitation [1][2]. Thompson Sampling maintains a probability distribution over expected rewards for each candidate action, samples from those distributions, and selects the action with the highest sampled value. When rewards are binary or near-binary, the Beta-Bernoulli conjugate model allows for efficient Bayesian posterior updates. Agrawal and Goyal demonstrated that Thompson Sampling achieves near-optimal cumulative regret in stochastic bandit settings [15], making it an attractive choice for adaptive policy routing in document processing.

A practical difficulty in deploying bandit algorithms is the cold-start problem, where a newly encountered document context has no prior experience to draw upon. One approach to mitigating this problem is to transfer knowledge from similar past experiences using retrieval-augmented methods. Sentence embedding models such as Sentence-BERT [5] encode textual context into dense vector representations that support cosine similarity search, enabling the identification of related past experiences even when exact matches are unavailable. Paired with vector databases such as pgvector for PostgreSQL, these embeddings support efficient nearest-neighbor lookups at scale.

Concept drift, defined as a change in the statistical properties of model inputs over time, poses an additional challenge for adaptive systems. Document processing environments may see new vendors, updated invoice formats, or changing regulatory requirements that gradually alter the effective performance of learned policies. Ebbinghaus's foundational work on memory and forgetting [14] established that information not reinforced decays over time, an insight that has been formalized in machine learning through adaptive forgetting mechanisms. Dual sliding-window approaches to drift detection compare recent performance against older observations to identify when a significant shift has occurred, triggering targeted updates to the model.

On the extraction side, large language models have demonstrated remarkable capability for structured information extraction from unstructured text [3]. The Transformer architecture introduced by Vaswani et al. [12] with its self-attention mechanism forms the backbone of modern language models, and encoder-decoder architectures such as BART [3] extend this to sequence-to-sequence tasks including question answering and summarization. Local Optical Character Recognition (OCR) engines such as PaddleOCR [9] provide the text extraction layer that converts document images into processable text prior to LLM-based field extraction.

Hallucination, the generation of factually incorrect content by language models, remains a significant reliability concern in document AI [3]. Traditional evaluation metrics such as F1 score measure extraction accuracy against ground truth labels but

do not detect hallucinated values that are absent from the source document. Faithfulness-based evaluation approaches compare extracted values against the source text using semantic similarity to identify grounding failures. Personal Information Protection is a further concern; Presidio, Microsoft's open-source de-identification toolkit [10], provides entity recognition for sensitive attributes including credit card numbers, social security numbers, and medical license numbers.

Explainability is essential for deploying AI systems in high-stakes environments such as healthcare and finance, where decisions must be auditable and understandable by human reviewers. LIME (Local Interpretable Model-agnostic Explanations) [4] addresses this by training a local linear model around individual predictions, assigning importance scores to input tokens based on their influence on model output. Cross-attention mechanisms in encoder-decoder architectures provide an alternative form of attribution by revealing which encoder tokens the decoder attended to when generating each output token. Human-in-the-loop systems formalize the role of human reviewers by capturing structured feedback on model outputs and using that feedback to guide future behavior.

The microservices architectural pattern provides the deployment foundation for complex AI platforms. Docker containers [7] enable reproducible, isolated service deployments with predictable resource consumption, while REST APIs provide a standardized interface for inter-service communication. JSON Web Tokens (JWT) offer a stateless mechanism for authentication across distributed services. BM25 [6], a term-frequency-based retrieval function, complements vector similarity search by capturing keyword-level matches that dense embeddings may miss, and hybrid retrieval systems combining both approaches have demonstrated improved recall in document retrieval tasks.

1.2 Research Gap

Despite substantial progress in individual areas of document AI, several gaps remain in existing literature. First, adaptive policy selection for document processing is rarely treated as a formal sequential decision problem. Most commercial systems apply fixed processing pipelines configured by human engineers, providing no mechanism for the system to learn which strategy works best for each document class based on accumulated experience.

Second, the cold-start problem in bandit-based document processing has not been addressed systematically. When a document type is encountered for the first time, existing systems either fall back to a generic default policy or require manual configuration. A principled mechanism for transferring relevant prior experience to cold-start contexts is lacking.

Third, integrated systems that combine adaptive policy routing with multi-layer hallucination detection, structured explainability, and human-in-the-loop correction are absent from the literature. Existing systems treat these concerns separately, requiring significant integration effort and producing fragmented operational experiences. A unified platform that addresses all four concerns within a single cohesive architecture is therefore needed.

Fourth, concept drift in document processing environments has received limited attention. As vendor formats evolve and new document types are introduced, the optimal processing strategy for a given context class may change over time. Systems that do not account for drift will gradually degrade in performance without any visible signal to the operator.

1.3 Research Problem

The central research problem addressed in this dissertation is: how can a document AI platform be designed so that it adaptively selects and refines processing policies based on accumulated experience, reliably detects and prevents hallucinated outputs, and provides transparent, auditable explanations of its decisions to human reviewers, all within a scalable microservices deployment?

1.4 Research Objectives

The following specific objectives were defined to address the research problem:

- (i) To design and implement an Adaptive Hierarchical Memory Network (AHMN) that uses Thompson Sampling with Beta-Bernoulli posteriors for document processing policy selection, incorporating a novel Retrieval-Augmented Bandit Initialization mechanism for cold-start knowledge transfer and a Drift-Aware Adaptive Forgetting mechanism for handling concept drift.
- (ii) To develop an Intelligent Document Processing Orchestrator (IDPO) that executes a thirteen-agent pipeline driven by policies recommended by the AHMN, supporting PaddleOCR and Tesseract for text extraction, LLM-based field extraction via local Ollama models, and hybrid BM25 and vector retrieval for question answering.
- (iii) To implement a six-layer Guardrail Ensemble for hallucination detection and output validation, covering schema adherence, semantic relevance, internal consistency, domain rules, PII leakage detection, and weighted faithfulness scoring.
- (iv) To build an Explainable AI system with Human-in-the-Loop (XAI-HITL) feedback, providing BART-based generative question answering with cross-attention and LIME-based explanations and a structured feedback mechanism for human reviewers.
- (v) To deploy the complete platform as eight containerized Docker microservices with network isolation, JWT-secured API gateway, and PostgreSQL with pgvector for persistent storage and vector similarity search.

2. METHODOLOGY

2.1 System Architecture Overview

The platform is implemented as eight Docker services communicating over two isolated Docker bridge networks. An external network exposes the Frontend and Gateway to the public internet and permits outbound access required for model downloads from HuggingFace. An internal network, configured with the Docker internal flag set to true, connects the four AI microservices with the database and cache tiers, ensuring that no internal service is reachable from the internet directly. All inter-service communication over the internal network uses service-discovery DNS names defined in the Docker Compose configuration.

The eight services are the Frontend (Next.js, port 3000), the API Gateway (FastAPI, port 8000), the AHMN (FastAPI, port 8001), the Guardrail service (FastAPI, port 8002), the XAI-HITL service (FastAPI, port 8003), the IDPO (FastAPI, port 8004), PostgreSQL with pgvector (port 5432), and Redis (port 6379). Table 1 shows the resource allocations for each service as configured in the Docker Compose file.

Service	Memory Limit	CPU Limit	Healthcheck Start Period
Database (pgvector)	1 GB	1.0	—
Redis	256 MB	0.5	—
AHMN	1 GB	1.0	90 s
Guardrails	1.5 GB	0.5	120 s
XAI-HITL	4 GB	2.0	180 s
IDPO	1 GB	1.0	30 s
Gateway	512 MB	0.5	30 s
Frontend	512 MB	0.5	30 s

Table 1 - Docker service resource allocations

The XAI-HITL service is allocated 4 GB of memory and 2.0 CPU units because it loads the BART encoder-decoder model and the spaCy Named Entity Recognition model at startup; both models are held in memory throughout the service lifetime to avoid repeated loading overhead. The Guardrail service at 1.5 GB accommodates the SentenceTransformer model used for semantic similarity scoring and the Presidio analyzer for PII detection. The remaining services operate within 1 GB each, with the database and cache tiers sized for their respective workloads.

A request to process a document flow from the Frontend through the Gateway to the IDPO. The IDPO first consults the AHMN to obtain a policy recommendation, executes the thirteen-agent pipeline using that policy, then logs the outcome back to the AHMN for posterior updates. Guardrail validation may be triggered either inline within the IDPO pipeline or as a standalone request from the Frontend. Question-answering requests are routed by the Gateway directly to the XAI-HITL service.

2.2 Database Design

The system uses PostgreSQL 16 as its primary persistent store, extended with the pgvector extension to support vector similarity search over 384-dimensional embeddings. Redis is used as a session-scoped cache for Thompson Sampling posteriors and as a token blacklist for revoked JWT refresh tokens. The database schema is initialized from a single SQL file and comprises seven tables as summarized in Table 2.

Table Name	Purpose
users	User accounts with bcrypt-hashed passwords and role-based access
password_reset_tokens	Time-limited tokens for self-service password reset
mtm_outcomes	Mid-term memory: raw outcome logs with document context and performance metrics
ltm_validated_policies	Long-term memory: promoted strategies with 384-d vector embeddings and Beta posteriors
experiment_runs	Experiment tracking for bandit algorithm comparison
guardrail_audit_log	Persistent audit trail for all validation decisions
xai_hitl_feedback	Human reviewer feedback on Q&A answers and XAI explanations
system_events	General-purpose system event log

Table 2 -Database tables and their purpose

The `mtm_outcomes` table stores a JSONB `document_context` column containing the document characteristics (document type, vendor name, content profile, quality bin, and routing metadata) alongside an `outcome_metrics` JSONB column holding the blended reward signal components. This design allows flexible schema evolution without requiring table migrations as new context or metric fields are added.

The `ltm_validated_policies` table is the most structurally important table in the AHMN tier. It stores a `context_embedding` column of type vector (384), indexed with an inner-product operator class provided by `pgvector`, enabling approximate nearest-neighbour queries in sub-linear time. The `alpha` and `beta` columns hold the Beta distribution parameters for the Thompson Sampling posterior of each validated policy, and the `confidence_score` column provides a composite measure used during policy retrieval.

2.3 Adaptive Hierarchical Memory Network (AHMN)

The AHMN is the adaptive core of the platform. Its responsibility is to recommend which document processing policy should be applied to a given document context, and to update its internal model based on observed outcomes. It implements a three-tier memory architecture inspired by cognitive models of human memory, using Redis for short-term session state, PostgreSQL for mid-term outcome logs, and PostgreSQL with `pgvector` for long-term validated knowledge.

2.3.1 Three-Tier Memory Architecture

Tier	Storage	Purpose	Retention Policy
STM	Redis	Session-scoped Beta posteriors (α , β per policy per context)	TTL-based; expires with session
MTM	PostgreSQL mtm_outcomes	Raw outcome logs with document context and metrics	Permanent; source of truth for promotion
LTM	PostgreSQL ltm_validated_policies + pgvector	Promoted strategies with 384-d embeddings and Beta posteriors	Decayed by DAAF forgetting; archived when inactive

Table 3 - AHMN three-tier memory architecture

When a document arrives, the AHMN first checks the STM for an existing posterior for the current context. If found, it samples from those STM posteriors. If not found, it initiates the RABI cold-start procedure to initialize posteriors from the LTM. The selected policy is returned to the IDPO, which uses it to configure the processing pipeline. After the pipeline completes, the IDPO logs the outcome to the AHMN, which updates the STM posteriors immediately and asynchronously queues an MTM write.

2.3.2 Thompson Sampling with Beta-Bernoulli Posteriors

Policy selection is performed using Thompson Sampling with Beta-Bernoulli conjugate posteriors [1][2]. For each candidate policy p , the algorithm samples a scalar value from its Beta distribution: $\theta_p \sim \text{Beta}(\alpha_p, \beta_p)$. The policy with the highest sampled value is selected. This probabilistic selection naturally balances exploration of less-tested policies against exploitation of high-performing ones.

Posterior updates follow a simple rule: on a successful outcome (where the blended reward signal meets or exceeds a configurable threshold), α is incremented by 1.0; on a failed outcome, β is incremented by 1.0. When a human reviewer marks an answer as correct, α receives an additional increment of 1.0 as a reinforcement signal from the HITL feedback loop.

Event	Update Rule
Successful outcome (reward $\geq$ threshold)	$\alpha \leftarrow \alpha + 1.0$
Failed outcome (reward $<$ threshold)	$\beta \leftarrow \beta + 1.0$
Human feedback: correct	$\alpha \leftarrow \alpha + 1.0$ (additional boost)

Table 4 - Thompson Sampling posterior update rules

The blended reward signal is a weighted combination of multiple quality indicators rather than pure extraction F1 score. Specifically, it incorporates the F1 score (when ground truth labels are available), schema validity as a binary indicator, a grounding score measuring the proportion of extracted values that can be traced to the source text, a hallucination penalty applied when the Guardrail service flags a hallucination, and a latency penalty applied when pipeline processing time exceeds a configurable threshold. These weights are stored in the service configuration and can be adjusted without redeployment.

2.3.3 RABI: Retrieval-Augmented Bandit Initialization

The RABI mechanism addresses the cold-start problem that arises when a document context is encountered for the first time and no STM posteriors exist. Rather than falling back to uninformative prior distributions ($\alpha=1$, $\beta=1$), RABI queries the LTM for semantically similar past contexts and transfers their accumulated knowledge to initialize the new posteriors.

Context representation is performed by the embedding service, which encodes a structured context string into a 384-dimensional vector using the all-MiniLM-L6-v2 SentenceTransformer model [5]. The context string incorporates document type, vendor name, content profile (e.g., text-heavy, table-heavy), layout cluster identifier, quality bin, and structural features such as table ratio, form ratio, key-value density, and numeric density. This rich feature representation allows the embedding to capture document characteristics that determine which processing policy is most suitable.

Retrieval is performed by querying pgvector for LTM entries whose embeddings have cosine similarity above a minimum threshold of 0.4. Neighbor weights are computed

using a softmax function with temperature $\tau=0.1$: $w_i = \text{softmax}(\text{sim}_i / \tau)$. The pseudocount transfer formula is: $\alpha_{\text{transfer}} = \sum w_i \times (\alpha_i - 1)$, $\beta_{\text{transfer}} = \sum w_i \times (\beta_i - 1)$, with each transfer capped at a maximum of 10.0. The new context is initialized with $\alpha_{\text{new}} = 1.0 + \alpha_{\text{transfer}}$ and $\beta_{\text{new}} = 1.0 + \beta_{\text{transfer}}$, reflecting the degree of confidence borrowed from similar historical contexts.

Parameter	Value	Description
similarity_threshold	0.4	Minimum cosine similarity for neighbour inclusion
temperature (τ)	0.1	Softmax temperature for weight computation
max_transfer	10.0	Maximum pseudocount transfer per α or β
transfer_decay	0.95	Per-round decay factor for transferred pseudocounts

Table 5 - RABl cold-start transfer parameters

To ensure that the agent gradually transitions from relying on transferred knowledge to its own observations, the transferred pseudocount are decayed each round by a factor of 0.95. This decay prevents the agent from remaining over-anchored to historical policies that may not generalize well to the specific characteristics of the new context.

2.3.4 DAAF: Drift-Aware Adaptive Forgetting

The DAAF mechanism addresses concept drift in the document processing environment. It operates on a dual sliding-window basis, comparing recent policy rewards against older rewards to detect whether a statistically significant shift has occurred. Two drift severity levels are defined: gradual drift (a smaller, slower shift) and abrupt drift (a larger, faster shift).

Drift detection uses two criteria applied in conjunctions. The first is a magnitude threshold on the mean reward difference Δ between the newer and older windows. The second is a Welch z-test that accounts for unequal variance between the two windows: $z = \Delta / \sqrt{(\sigma^2_{\text{old}}/n_{\text{old}} + \sigma^2_{\text{new}}/n_{\text{new}})}$. Abrupt drift is declared when $|\Delta| \geq 0.15$ or $z \geq 3.0$; gradual drift is declared when $|\Delta| \geq 0.05$ or $z \geq 2.0$.

Parameter	Value
Window size	50 observations
Minimum samples before detection	5
Gradual drift magnitude threshold	0.05
Abrupt drift magnitude threshold	0.15
Gradual drift z-test threshold	2.0
Abrupt drift z-test threshold	3.0

Table 6 - DAAF drift detection parameters

When drift is detected, the optimal forgetting rate is computed as: $\gamma^* = \sqrt{(\Delta^2 / \ln(t))}$, where t is the total observation count. The DAAF scheduler applies this forgetting rate to LTM entries in a batch process that runs every six hours. Policies that have been inactive for more than seven days and whose confidence score falls below 0.35 after applying the forgetting rate are archived and removed from the active LTM. Promotion from MTM to LTM runs every thirty minutes and applies the criteria of F1 score above 0.50, at least two outcome observations, and the policy must not be the default fallback policy.

2.4 Intelligent Document Processing Orchestrator (IDPO)

The IDPO orchestrates the complete document processing pipeline through thirteen specialized agents arranged in a structured execution sequence. The pipeline is controlled by three layers: the Orchestrator, which coordinates the overall workflow; the PlanRouter, which integrates with the AHMN to select a policy; and the PlanExecutor, which runs each agent in turn according to the execution plan produced by the selected policy.

2.4.1 Thirteen-Agent Pipeline

#	Agent	Stage Operator	Purpose
1	DocumentAnalysisAgent	Pre-pipeline	Compute quality_bin, doc_type, vendor_cluster, content_profile
2	OcrAgent	preprocess.ocr	PaddleOCR or Tesseract OCR based on policy configuration
3	DeskewAgent	preprocess.deskew	Correct rotation and skew in scanned document images
4	DenoiseAgent	preprocess.denoise	Apply noise reduction to improve OCR quality
5	TableFormAgent	preprocess.table_extract	Detect table and form structures in document layout
6	LLMExtractionAgent	structuring.extract	LLM-based field extraction using Ollama or OpenAI-compatible API
7	SectioningAgent	structuring.section	Divide document into logical sections
8	SummarizationAgent	structuring.summary	Produce document-level summary
9	ValidationAgent	validation	Validate extracted fields and compute grounding score
10	RecoveryAgent	recovery	Re-extract fields that failed initial validation
11	EscalationAgent	escalation	Escalate issues that cannot be resolved by recovery
12	ChunkingAgent	chunking	Segment document into retrieval chunks
13	AnsweringAgent	—	Answer questions over document chunks using retrieved context

Table 7 - IDPO thirteen-agent pipeline

The DocumentAnalysisAgent runs before the main pipeline and produces a DAEOutput object containing the document's quality bin, document type classification, vendor cluster identifier, and content profile. This output is used by the PlanRouter to construct the AHMN advice payload and by subsequent agents to adapt their behaviour to the specific document characteristics.

The LLMExtractionAgent supports three extraction modes. In bulk extraction mode, a single LLM call is made with all required fields listed in the prompt. In per-field extraction mode, one LLM call is made for each field, which improves precision for complex documents at the cost of higher latency. In thinking mode, the Qwen3 model's

chain-of-thought reasoning capability is engaged for documents that require multi-step inference. The active mode is determined by the policy configuration recommended by the AHMN.

The ValidationAgent performs two key checks. First, it validates extracted field values against type-specific rules (e.g., numeric fields must be parseable numbers, dates must follow expected formats). Second, it computes a grounding score by searching for each extracted value in the OCR text of the source document, providing a proxy for faithfulness that does not require ground truth labels.

The RecoveryAgent attempts to recover fields that failed validation by applying alternative extraction strategies, including heuristic pattern matching and generic key-value extraction. If recovery is unsuccessful, the EscalationAgent marks the document for human review. This three-stage structure (extraction, validation, recovery) ensures that the pipeline makes every reasonable attempt to produce a complete result before escalating.

2.4.2 Retrieval System

The IDPO maintains a hybrid retrieval index for each document, combining a BM25 keyword index [6] with a vector store based on a custom HashingEmbedder that produces 256-dimensional embeddings. The BM25 index handles precise keyword lookups effectively for financial terms and entity names, while the vector store captures semantic similarity for questions phrased differently from the source text. The combined IndexBundle is used by the AnsweringAgent and the retrieval-augmented extraction pathway to ground LLM inputs in relevant document chunks.

2.4.3 Provenance and Audit Trail

Every agent in the pipeline records a provenance event through the ProvenanceRecorder, logging its inputs, outputs, configuration parameters, and references to stored artefacts. The ArtifactStore maintains versioned copies of intermediate pipeline outputs including raw OCR text, execution plans, extracted entity sets, and document chunks. This comprehensive audit trail supports post-hoc analysis of pipeline decisions and provides the data needed to diagnose extraction failures.

2.5 Guardrail Ensemble

The Guardrail service implements a six-layer validation ensemble that assesses the quality and safety of extracted document data. The layers execute sequentially with error isolation, meaning a failure in one layer's computation does not prevent the remaining layers from running. A document passes the ensemble only if all six layers pass.

#	Check	Score Range	Pass Condition	Description
1	Schema Adherence	0.0 / 1.0	score = 1.0	Pydantic model validation per document type; checks required fields and data types
2	Relevance	0.0 – 1.0	avg $\geq$ 0.50	Cosine similarity between extracted field values and source text using SentenceTransformer
3	Internal Consistency	0.0 / 1.0	No errors	Business logic: arithmetic checks, negative value detection, future date detection, dosage validation
4	Domain Rules	0.0 / 0.5 / 1.0	No errors	Tax plausibility, vendor fraud detection, suspicious keyword detection, large-amount thresholds
5	PII Leakage	0.0 – 1.0	Context-dependent	Presidio-based PII detection with per-document-type allowlists and blocklists
6	Faithfulness	0.0 – 1.0	score $\geq$ 0.62	Weighted combination of semantic, numerical, and entity matching against source text

Table 8 - Guardrail ensemble validation layers

2.5.1 Schema Adherence

The Guardrail service maintains Pydantic model definitions for eight document types: ClinicalSummary, MedicationDosage, Invoice, Receipt, BankStatement, PurchaseOrder, InsuranceClaim, and a generic fallback. Each model specifies which fields are required and which are optional and enforces type constraints on field values. Unexpected fields are allowed for financial document types where vendor-specific fields are common but are penalised in the Relevance layer. Clinical document types have stricter required-field enforcement because missing fields represent patient safety risks.

2.5.2 Relevance Check

The Relevance check detects hallucinated field values by computing the cosine similarity between each extracted value and the full source text, using the all-MiniLM-L6-v2 SentenceTransformer model [5]. Values with similarity below a medium threshold of 0.35 are classified as potential hallucinations. Known financial keywords such as "total", "subtotal", "tax", and "balance" receive a similarity boost of +0.35 because they frequently appear as table headers in structured documents, which may reduce their raw cosine similarity with source text even when correctly extracted. Fields not specified in the document type's schema receive a penalty multiplier of 0.4.

2.5.3 Internal Consistency

The Internal Consistency layer applies document-type-specific business logic rules. For invoices, it verifies that subtotal plus tax equals total within a tolerance of 0.05. For receipts, it checks that the total is non-negative, the merchant name is not empty, and the date is not in the future. For clinical summaries, it checks that the condition field is non-empty. For medication dosages, it verifies that dosage_mg is positive and that the frequency field contains a recognized value. These checks catch arithmetic errors and logical contradictions that semantic similarity alone cannot detect.

2.5.4 Domain Rules

The Domain Rules layer applies heuristics that reflect real-world knowledge about document content. Keyword-based fraud detection flags documents containing values such as "dummy", "fake", "test data", or "demo data". Tax plausibility checking flags invoice tax rates outside the range of 0% to 30%, consistent with the Sri Lanka Value Added Tax standard rate of 18% in 2026. Vendor fraud detection compares the customer name against the vendor name on invoices and raises a flag if they match. Large-amount thresholds flag invoices above LKR 1,000,000, receipts above LKR 50,000, purchase orders above LKR 5,000,000, and insurance claims above LKR 1,000,000 for additional review.

2.5.5 PII Leakage Detection

PII detection is performed using the Presidio AnalyzerEngine [10], which identifies personal information entities including person names, phone numbers, credit card numbers, national registration numbers, social security numbers, IBAN codes, and medical license numbers. Context-aware rules determine which entity types are acceptable in each document context. Medical documents may legitimately contain patient names and dates; business documents may contain names, phone numbers, and national registration numbers. Credit card numbers, IBAN codes, and social security numbers are flagged as leakage in all contexts.

2.5.6 Faithfulness Scoring

The Faithfulness check provides a composite score that combines three components: semantic similarity between extracted field values and source text, numerical match between extracted amounts and numbers present in the source document, and an entity matching score. Table 2.10 shows the component weights.

Component	Weight	Method
Semantic similarity	0.50	Cosine similarity with SentenceTransformer; weighted by field importance
Numerical match	0.40	Exact numeric matching between extracted amounts and source text numbers
Entity match	0.10	Entity-level match score (default 0.85 when entity extraction unavailable)

Table 9 - Faithfulness scoring component weights

Field importance weights modulate the contribution of each field to the semantic component. High-importance fields include "total" (weight 1.5×), "claim_amount" (1.5×), "subtotal" (1.4×), "balance" (1.4×), and "tax" or "vat" (1.3×). All other fields use a weight of 1.0×. This weighting reflects the fact that errors in financial totals are more consequential than errors in lower-importance descriptive fields. The faithfulness threshold for passing is 0.62.

2.6 Explainable AI and Human-in-the-Loop (XAI-HITL)

The XAI-HITL service provides two core capabilities: generative question answering over processed document content, and structured explainability mechanisms that allow human reviewers to understand and verify the system's answers. It is the largest service in the platform in terms of memory (4 GB) due to the BART language model that it loads at startup.

2.6.1 BART-based Generative Question Answering

The service uses the vblagoje/bart_lfqa model from HuggingFace Transformers, an encoder-decoder architecture based on BART [3] fine-tuned for long-form question answering. Input is formatted as "question: Q} context: {C}" and fed to the model, which generates an answer using beam search with 4 beams. The no-repeat n-gram size is set to 3 to reduce repetition, the maximum input length is 1,024 tokens, and the maximum output length is 128 tokens. The attention implementation is set to "eager"

mode because PyTorch's flash attention implementation does not expose the cross-attention tensors needed for the XAI pipeline.

Parameter	Value
Base model	vblagoje/bart_lfqa (HuggingFace Transformers)
Max input length	1,024 tokens
Max output length	128 tokens
Beam search width	4 beams
No-repeat n-gram size	3
Attention implementation	eager (required for output_attentions=True)
Confidence calculation	$\exp(\text{sequence_log_probability})$, normalised to [0,1]

Table 10 - BART model configuration parameters

The Q&A pipeline includes several post-generation validation steps. An unanswerable detector uses keyword-based matching and content validation to identify cases where the model cannot find a relevant answer in the provided context. An AnswerValidator checks that any numeric values in the generated answer can be found in the source text, preventing digit substitution hallucinations. A digit-swap detector identifies cases where the model generates a number that uses the same digits as a source number but in a different order.

2.6.2 Question Type Handling

Type	Detection	Handling Strategy
Simple Q&A	Default	Direct BART generation with full validation pipeline
Yes/No	Question starts with "is", "are", "does", "did"	Conversational synthesis integrating Boolean answer with supporting evidence
List questions	"list all", "what are the" + NER entity detection	spaCy NER extraction for organisation entities; combines detected names
Comparison	"highest", "lowest", "most", "least"	Question decomposition into per-entity sub-queries; numeric comparison applied to results
Financial	Amount/tax/total keywords in question	TableReconstructor pre-processing; AnswerValidator post-validation of numeric values

Table 11 - XAI-HITL question type handling

Comparison questions undergo a decomposition procedure. The service first detects the comparison type (highest, lowest, most, least) and the target attribute from the question. It then uses spaCy's named entity recognition to extract candidate entities from the context. A sub-question is generated for each entity, BART is called for each sub-question, numeric values are extracted from the answers, and the comparison logic selects the winning entity. A `DecomposedExplanation` object is constructed that includes XAI output for each sub-result.

2.6.3 Explainability Mechanisms

Two complementary XAI mechanisms are implemented. The first is cross-attention extraction, which is referred to as SHAP in the external API for compatibility with client expectations but is implemented as decoder-to-encoder cross-attention from BART's last transformer layer. Attention weights are averaged across all attention heads and all decoder positions to produce per-encoder-token importance scores. These scores are mapped back to character offsets in the original context text to enable highlighting in the frontend interface.

The second mechanism is LIME [4], implemented using the LimeTextExplainer from the lime library. Rather than using classification confidence as the prediction function, the LIME predictor uses the sequence probability of the target answer given a perturbed context: $P(\text{answer} \mid \text{context}', \text{question}) = \exp(-\text{cross_entropy_loss})$. LIME perturbs the context text by masking words, generates between 15 and 20 samples depending on document length, fits a local linear model to the probability changes, and returns per-word importance coefficients that indicate each word's contribution to the answer.

The service supports three XAI operating modes: fast mode (`include_xai=False`), in which only the answer is generated with a 120-second timeout; full XAI mode (`include_xai=True`), which computes both cross-attention and LIME explanations with a 180-second timeout; and lazy XAI mode via the `/xai/explain` endpoint, which computes explanations for a previously generated known answer on demand with a 300-second timeout. This progressive disclosure approach minimises latency for routine queries while making full explanations available when needed.

2.6.4 HITL Feedback System

Human reviewers can submit structured feedback on any answer through the HITL feedback endpoint. Feedback types are: "correct", "incorrect", "partially_correct", and "needs_more_context". Reviewers also provide 1-to-5 scale ratings for explanation quality, highlight accuracy, and overall XAI helpfulness. Correct feedback triggers an α increment in the AHMN for the policy that processed the relevant document, creating a closed feedback loop between human quality assessments and the bandit policy selection mechanism. Feedback is persisted to PostgreSQL asynchronously and hydrated into an in-memory deque on service startup for fast read access.

2.7 API Gateway

The Gateway service is the single-entry point for all external requests. It performs JWT authentication, rate limiting at 200 requests per minute per IP using SlowAPI backed by Redis, and transparent proxying of requests to the appropriate internal microservice using the httpx async HTTP client library. The Gateway handles authentication locally using PostgreSQL and does not forward authentication requests to any downstream service.

JWT access tokens have a 24-hour time-to-live and are signed using the HS256 algorithm. Refresh tokens have a 7-day time-to-live and are rotated on each refresh, with the old token blacklisted in Redis using a "revoked:" key prefix. Passwords are hashed using bcrypt before storage. The Gateway also exposes a deep health check endpoint that aggregates the health status of all eight services, providing a single operational status view.

Proxy timeouts are configured differently for the IDPO versus other services. Standard read timeout is 300 seconds for most services. The IDPO read timeout is extended to 1,200 seconds (20 minutes) to accommodate long-running document processing jobs on large or complex documents. Write timeout is 120 seconds for all services.

2.8 Frontend Dashboard

The Frontend is implemented in Next.js (React) with Tailwind CSS for styling. It supports both dark and light modes through a theme provider component and uses JWT middleware for authentication, automatically refreshing tokens before expiry. The dashboard is structured into sections corresponding to each microservice: an AHMN section exposing the DAAF forgetting visualization, the RABI cold-start visualization, and the experiment tracking view; a Guardrail section showing the validation dashboard with recent audit log entries; an XAI-HITL section providing the Q&A interface, detailed explanation view, and HITL feedback management; and an IDPO section offering a benchmark runner, run comparison view, and job management interface.

2.9 Deployment and Infrastructure

The complete platform is defined in a single Docker Compose file that specifies all eight services, their resource limits, health checks, restart policies, and network memberships. Services are connected to the internal Docker bridge network for inter-service communication and those requiring internet access (for downloading HuggingFace models during first startup) are additionally connected to the external bridge network. The database and Redis services are connected only to the internal network and have no external exposure.

The IDPO uses locally hosted LLM inference via Ollama, supporting the Qwen3.5:9b model for high-accuracy extraction and the phi3:mini model as a lighter-weight alternative. LLM inference is performed on the host GPU when available, with CPU fallback. The SentenceTransformer model (all-MiniLM-L6-v2) and the Presidio Analyzer model are downloaded from the internet on first startup and cached locally. Table 12 summarizes the ML models used across all services.

Service	Model	Library	Purpose
AHMN	all-MiniLM-L6-v2	SentenceTransformers	384-d context embeddings for LTM similarity search
Guardrails	all-MiniLM-L6-v2	SentenceTransformers	Semantic similarity for relevance and faithfulness scoring
Guardrails	Presidio Analyzer	Microsoft Presidio	PII entity detection and leakage flagging
XAI-HITL	vblagoje/bart_lfqa	HuggingFace Transformers	Generative Q&A (encoder-decoder)
XAI-HITL	en_core_web_sm	spaCy	Named Entity Recognition for list and comparison questions
IDPO	qwen3.5:9b / phi3:mini	Ollama (local)	LLM-based structured field extraction
IDPO	HashingEmbedder (custom)	Custom (dim=256)	Document chunk retrieval in hybrid BM25+vector index

Table 12 - ML models used across services

3. RESULTS AND DISCUSSION

3.1 System Integration

The platform was successfully deployed as eight Docker microservices operating over isolated internal and external networks. All services passed their health check probes within the configured start periods, with the XAI-HITL service requiring the longest startup time of 180 seconds due to the loading and warm-up of the BART model. Inter-service communication over the internal Docker bridge network was verified through end-to-end document processing requests that exercised all major request flows: document submission, AHMN policy advice, pipeline execution, guardrail validation, and Q&A with XAI.

The Gateway rate limiter was tested by submitting rapid-burst requests at volumes exceeding 200 per minute per IP; the limiter correctly applied HTTP 429 responses to excess requests while maintaining normal throughput for traffic within the limit. JWT token rotation was verified by confirming that using a blacklisted refresh token after a token refresh cycle returned an authentication error, as expected.

3.2 AHMN Policy Adaptation

The AHMN was evaluated by submitting sequences of documents from multiple document types and observing the evolution of Thompson Sampling posteriors. For document contexts with consistent processing outcomes, the Alpha parameters of high-performing policies grew steadily, reflecting the accumulation of successful outcomes, while the Beta parameters of lower-performing policies grew more slowly. This behavior is consistent with the theoretical properties of Thompson Sampling, which concentrates sampling probability on high-performing arms as evidence accumulates [1][2].

The RABI cold-start mechanism was evaluated by submitting a document type not previously encountered in the current session. The mechanism successfully retrieved semantically similar LTM entries using pgvector similarity search and transferred their pseudocount to initialize the new context's posteriors. The use of the all-MiniLM-L6-v2 embedding model [5] with a 384-dimensional representation provided sufficient

discriminative capacity to distinguish between document contexts with different structural characteristics.

The DAAF forgetting mechanism was exercised by simulating a concept drift scenario in which the reward distribution for a policy shifted mid-sequence. The dual sliding-window detector identified the shift after accumulating sufficient evidence in both windows (minimum five samples), computed an optimal forgetting rate using the formula $\gamma^* = \sqrt{(\Delta^2 / \ln(t))}$, and applied the corresponding decay to the affected LTM entries. Policies whose confidence scores fell below the 0.35 threshold following decay were correctly archived and removed from the active retrieval pool.

3.3 Guardrail Validation Coverage

The Guardrail ensemble was tested against a set of documents spanning all eight supported document types, including cases with intentionally injected defects such as hallucinated total amounts, mismatched invoice arithmetic, future dates on receipts, and synthesized PII values in inappropriate contexts. The six-layer pipeline demonstrated layered coverage in that different defect types were caught by different layers.

Arithmetic inconsistencies in invoices (where subtotal plus tax did not match the reported total within the 0.05 tolerance) were reliably caught by the Internal Consistency layer. Hallucinated field values that were entirely absent from the source document were detected by the Relevance layer, which reported cosine similarities well below the 0.50 threshold for such values. Fraudulent test documents containing keywords such as "dummy" and "fake" were correctly flagged by the Domain Rules layer. The Faithfulness layer provided the most comprehensive coverage for numerical hallucinations, combining semantic and numerical matching to identify cases where a plausible sounding but incorrect amount had been extracted.

PII leakage detection correctly flagged credit card numbers and IBAN codes in business documents where such values are not expected, while correctly passing person names and date-time values that are legitimately present in clinical summaries. The context-aware allowlist and blocklist design of the PII check proved essential for avoiding false positives in medical document contexts.

3.4 XAI-HITL Feedback Loop

The XAI-HITL service was tested with a range of question types across financial and clinical document contexts. The fast Q&A mode (without XAI) consistently returned answers within the 120-second timeout for standard single-fact questions over documents of moderate length. Full XAI mode added additional latency for the LIME perturbation computation, which requires between 15 and 20 BART forward passes for word importance scoring.

Cross-attention visualization correctly highlighted the most relevant passages for simple factual questions such as "what is the total amount?" on invoice documents, with the attention weights concentrating on the rows of the invoice that contained the total field. LIME importance scores were consistent with cross-attention highlights in most cases, providing corroborating evidence for the model's reasoning. In cases where the two mechanisms diverged, this divergence itself proved informative, indicating potential over-reliance on surface-level lexical patterns by one mechanism.

Human feedback collected through the HITL interface persisted to PostgreSQL and correctly propagated to the AHMN as Alpha increments for the policies that had processed the corresponding documents. This feedback loop was verified by confirming that the posterior of the relevant policy shifted following a batch of "correct" feedback submissions, increasing the policy's sampling probability in subsequent Thompson Sampling draws.

3.5 Summary of Each Student's Contribution

Dasanayake D R L (IT22350800) was responsible for the design and implementation of the Adaptive Hierarchical Memory Network (AHMN), including the three-tier memory architecture, the Thompson Sampling posterior update logic, the RABI cold-start transfer mechanism, the DAAF drift detection and forgetting algorithm, the background scheduler, and the embedding service using all-MiniLM-L6-v2.

Thimuth N was responsible for the Intelligent Document Processing Orchestrator (IDPO), including the thirteen-agent pipeline architecture, the PlanRouter integration

with the AHMN, the LLM extraction modes using Ollama, the hybrid BM25 and vector retrieval system, the provenance recorder, and the artifact store.

Wijetunga W.L.A.T. (Thenuka) was responsible for the Guardrail Ensemble, including the six-layer validation pipeline design, the Pydantic schema definitions for all document types, the SentenceTransformer-based relevance and faithfulness scoring, the Presidio-based PII leakage detection, and the audit log persistence infrastructure.

Pathirana M.M. (Malith) was responsible for the XAI-HITL service, including the BART-based generative Q&A pipeline, the cross-attention extraction mechanism, the LIME word importance implementation, the specialized question type handlers for yes/no, list, comparison, and financial questions, the TableReconstructor and AnswerValidator utilities, and the HITL feedback collection and persistence system.

All four team members contributed jointly to the API Gateway design, the Docker Compose deployment configuration, the PostgreSQL database schema, the Next.js frontend dashboard, and the overall system integration and testing.

3.6 Discussion

The modular microservices architecture proved to be an effective design choice for this platform. Service boundaries aligned naturally with the functional separation of concerns, allowing each team member to develop and test their component independently using the published REST API contract as the integration interface. The use of FastAPI across all Python services provided consistent endpoint patterns, automatic OpenAPI documentation generation, and asynchronous request handling that is essential for services performing long-running ML inference.

The decision to use the Beta-Bernoulli conjugate model for Thompson Sampling, rather than a more general Gaussian or non-parametric approach, was justified by the near-binary nature of the reward signal (pass/fail validation outcomes) and the computational efficiency of conjugate updates, which do not require numerical optimization. The blended reward signal introduces a degree of complexity compared to a pure binary reward, but it captures important quality dimensions (latency, grounding, schema validity) that a pure success/fail signal would miss.

The RABI mechanism's reliance on cosine similarity over 384-dimensional embeddings introduces a dependency on the quality of the context representation. The rich feature set incorporated into the context string (document type, vendor cluster, content profile, structural ratios) proved important for discriminating between contexts that might otherwise appear similar at a high level. Ablation of individual features from the context string is an area of future investigation.

The sequential design of the Guardrail ensemble, where all six layers must pass, is conservative by design. In production environments with stricter availability requirements, a partial-pass mode in which documents failing only low-severity checks are processed with appropriate warnings may be preferable. The current all-or-nothing design prioritizes correctness over throughput, which is appropriate for the research context.

4. CONCLUSION

This dissertation has presented the design, implementation, and evaluation of a modular multi-agent document AI platform that addresses the challenges of adaptive processing policy selection, hallucination detection, and explainable question answering in a unified microservices architecture. The four main components, namely the AHMN, the IDPO, the Guardrail Ensemble, and the XAI-HITL service, were each implemented as independent Docker services communicating through a JWT-secured API Gateway, with PostgreSQL and pgvector providing the persistent storage and vector similarity search infrastructure.

The AHMN's Thompson Sampling mechanism with Beta-Bernoulli posteriors provided effective adaptive policy selection, with the RABI cold-start transfer mechanism successfully reducing the regret associated with novel document contexts by bootstrapping posteriors from semantically similar prior experiences. The DAAF drift detection and adaptive forgetting mechanism ensured that the system could respond to changes in document distribution without requiring manual intervention or full model retraining.

The thirteen-agent IDPO pipeline demonstrated reliable document extraction across a range of document types, with the policy-driven architecture enabling the system to select appropriate OCR engines, preprocessing steps, and LLM extraction modes for each document context. The hybrid BM25 and vector retrieval system provided a solid foundation for the question-answering pipeline, combining keyword precision with semantic recall.

The six-layer Guardrail Ensemble provided layered coverage against hallucination, arithmetic inconsistency, domain violations, and PII leakage, demonstrating that multi-layer validation is more robust than any single check applied in isolation. The BART-based XAI-HITL service provided accurate generative answers supported by cross-attention and LIME-based explanations that were consistent with each other in the majority of test cases, and the HITL feedback loop successfully propagated human quality signals back to the AHMN's Thompson Sampling posteriors.

4.1 Limitations

Several limitations of the current system should be acknowledged. First, the BART model used for Q&A was not fine-tuned on domain-specific document corpora, which limits its accuracy on highly technical or domain-specific questions. Second, the LTM seed set of 21 entries across 11 policies is relatively small; a larger, more diverse seed set would improve the quality of RABI cold-start transfers for uncommon document contexts. Third, the Guardrail ensemble's PII detection relies on Presidio's English-language entity models, which may have reduced accuracy for Sri Lankan names, addresses, and national identification formats. Fourth, the faithfulness scoring entity component currently uses a placeholder value of 0.85, which limits the precision of the faithfulness score for documents with complex entity structures.

4.2 Future Work

Several avenues for future development are identified. Fine-tuning the BART model on domain-specific document corpora, particularly for financial and clinical document types common in the Sri Lankan context, would be expected to improve Q&A accuracy substantially. Replacing the placeholder entity matching score in the faithfulness check with a production-quality NER pipeline would improve the precision of hallucination detection for entity-rich documents.

The DAAF mechanism's optimal forgetting rate formula could be extended to account for the heterogeneity of document types within a policy's reward history, applying per-type forgetting rates rather than a single aggregate rate. The RABI mechanism could be improved by incorporating auxiliary signals such as document layout cluster and quality bin directly into the pgvector similarity query using filtered nearest-neighbor search, reducing the risk of retrieving irrelevant neighbors with high embedding similarity.

Finally, extending the platform to support multi-modal document inputs, including PDF documents with embedded images and tables that require visual understanding beyond OCR text extraction, would significantly broaden its applicability. Vision-

language models such as those available through the HuggingFace Transformers library could be integrated as additional agent types within the IDPO pipeline.

REFERENCE

- [1] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933.
- [2] O. Chapelle and L. Li, "An empirical evaluation of Thompson sampling," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 24, 2011, pp. 2249–2257.
- [3] M. Lewis, Y. Liu, N. Goyal, M. Ghahraman, M. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, Online, 2020, pp. 7871–7880.
- [4] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 1135–1144.
- [5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, 2019, pp. 3982–3992.
- [6] S. E. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [7] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, Mar. 2014.
- [8] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017, pp. 4765–4774.
- [9] Y. Du, C. Chen, R. Jia, X. Tang, X. Lin, Y. Yuan, and Z. Li, "PP-OCR: A practical ultra lightweight OCR system," *arXiv preprint arXiv:2009.09941*, Sep. 2020.
- [10] Microsoft, "Presidio: Data protection and de-identification SDK," GitHub, 2022. [Online]. Available: <https://github.com/microsoft/presidio>. [Accessed: Apr. 25, 2026].
- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017, pp. 5998–6008.

- [12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional Transformers for language understanding," in Proc. 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT), Minneapolis, MN, USA, 2019, pp. 4171–4186.
- [13] S. Agrawal and N. Goyal, "Further optimal regret bounds for Thompson sampling," in Proc. 16th International Conference on Artificial Intelligence and Statistics (AISTATS), vol. 31, Scottsdale, AZ, USA, 2013, pp. 99–107.
- [14] J. M. J. Murre and J. Dros, "Replication and analysis of Ebbinghaus' forgetting curve," PLOS ONE, vol. 10, no. 7, p. e0120644, Jul. 2015.
- [15] R. T. Fielding and R. N. Taylor, "Principled design of the modern Web architecture," ACM Transactions on Internet Technology, vol. 2, no. 2, pp. 115–150, May 2002.

APPENDICES

Appendix A: Service API Endpoint Summary

Table A.1 provides a consolidated summary of the primary API endpoints exposed by each microservice through the Gateway. All endpoints listed under `/api/` require a valid JWT Bearer token in the Authorization header.

Table A.1: Consolidated API endpoint summary

Service	Method	Endpoint	Description
AHMN	POST	<code>/api/advice</code>	Get Thompson Sampling policy recommendation
AHMN	POST	<code>/api/advice/log</code>	Log pipeline outcome and update posteriors
AHMN	POST	<code>/api/advice/feedback</code>	Submit human reviewer feedback
AHMN	GET	<code>/api/dashboard/posteriors</code>	Retrieve current Beta posteriors
AHMN	GET	<code>/api/dashboard/ltm</code>	Retrieve LTM validated policies
Guardrails	POST	<code>/api/guardrails/validate</code>	Run 6-layer validation ensemble
Guardrails	GET	<code>/api/guardrails/history</code>	Recent audit log entries
XAI-HITL	POST	<code>/api/xai/ask</code>	Answer question with optional XAI
XAI-HITL	POST	<code>/api/xai/explain</code>	Generate XAI for known answer (lazy)
XAI-HITL	POST	<code>/api/hitl/feedback</code>	Submit human reviewer feedback
IDPO	POST	<code>/api/idpo/...</code>	Document processing and benchmark endpoints
Gateway	POST	<code>/api/auth/login</code>	User login (rate: 10/min)
Gateway	POST	<code>/api/auth/refresh</code>	Refresh JWT token pair
Gateway	GET	<code>/api/services/health</code>	All services health aggregation

Appendix B: Key Formulas Reference

B.1 Thompson Sampling selection: $\theta_p \sim \text{Beta}(\alpha_p, \beta_p)$; select $\arg \max_p \theta_p$

B.2 RABI weight: $w_i = \text{softmax}(\text{cosine}(q, e_i) / \tau)$

B.3 RABI transfer: $\alpha_{\text{new}} = 1.0 + \min(\sum w_i \times (\alpha_i - 1), \text{max_transfer})$

B.4 DAAF drift test: $z = \Delta / \sqrt{(\sigma^2_{\text{old}}/n_{\text{old}} + \sigma^2_{\text{new}}/n_{\text{new}})}$

B.5 DAAF optimal forgetting rate: $\gamma^* = \sqrt{(\Delta^2 / \ln(t))}$

B.6 Faithfulness: $\text{score} = 0.50 \times \text{semantic_avg} + 0.40 \times \text{numerical_match} + 0.10 \times \text{entity_score}$

B.7 BART confidence: $P = \exp(\text{sequence_log_probability})$

B.8 LIME sequence probability: $P(\text{answer} \mid \text{context}', \text{question}) = \exp(-\text{cross_entropy_loss})$